

Design Tradeoffs

Conway's Game of Life over int64 coordinates — reference for follow-up discussion, not required reading for the implementation itself. Companion to `README.md`.

1. Sparse vs. dense representation

2. String keys vs. nested maps

3. Packed representations

4. Arbitrary precision
5. Asymptotic complexity

6. Deterministic serialization

7. Streaming

8. Boundary semantics

9. Parser leniency

1. Sparse vs. dense representation

The board is stored sparsely: only live cells exist, as a `Set` of coordinate keys (`src/cell.ts`, `src/simulation.ts`). There is no 2D array and no bounding box allocated up front.

	Dense (flat/typed array)	Sparse (Set/Map of live cells)
Memory	$O(\text{board area})$ — fixed once a bound is chosen	$O(\text{live cells})$
Per-cell access	$O(1)$ via index arithmetic, low constant factor	$O(1)$ amortized via hashing, higher constant factor
Works with int64 coordinates?	No — a board spanning the full signed 64-bit range would need on the order of 2^{128} cells	Yes — cost tracks population, not coordinate range

A dense array is the better choice when the board is small and bounded (e.g. a fixed 100x100 grid): lower constant-factor overhead, cache-friendly, no hashing. That option doesn't exist here — the problem statement puts live cells anywhere in `$[-2^{63}, 2^{63} - 1]$` in each dimension, so any fixed-size array is either wrong (too small) or physically impossible (sized to the full range). Sparse is not being chosen for its own sake; it is the only representation whose cost is a function of the input rather than of the problem's stated coordinate domain.

Verdict: *Sparse storage, unconditionally — the coordinate range removes dense storage as an option rather than making it merely suboptimal.*

2. String keys vs. nested maps

Live cells are keyed by a template-literal string, ``${x},${y}`` (`src/cell.ts`), rather than a nested structure such as `Map<bigint, Set<bigint>>` (outer map keyed by `x`, inner set of `y` values present at that `x`).

	Nested Map/Set	Flat string key
Lookup for one coordinate	Two hash lookups (outer, then inner)	One hash lookup, but on a longer key (string concatenation cost)

Iteration	Nested loop over outer then inner	Single flat loop
Row-local operations (e.g. "all live cells at this x")	Cheap — direct access to the inner set	Not supported without a full scan
Implementation complexity	Two collections to keep in sync, more edge cases (empty inner sets after deletion)	One collection, one encode/decode pair

The nested form pays off when the workload does row-local queries (e.g. "is any cell alive in this horizontal band") or when many cells cluster in the same row, since the inner set amortizes the outer lookup. Neither applies here: `step()` only ever needs single-coordinate membership tests and a flat iteration over all live cells once per generation. The flat string key is simpler to reason about, requires no synchronization between two collections, and the encode/decode round-trip is a one-line `split(",") + BigInt(...)`.

Verdict: *Flat string key — the workload here is single-coordinate lookups and one full-set iteration per generation, exactly what a flat `Set` is good at; the nested structure's advantage (row-local queries) is never exercised.*

3. Packed representations

An alternative to a string key is a packed numeric key: interleave or concatenate the bits of `x` and `y` into a single `bigint`, e.g. `(x << 64n) | (y & MASK_64)` (treating each coordinate as an unsigned 64-bit field after a bias offset).

- **Potential upside:** a single 128-bit integer likely hashes and compares faster than a variable-length string built by template-literal concatenation, and avoids the decimal-to-bigint text parsing on decode.
- **Cost:** packing/unpacking needs an explicit bias (bigints don't have fixed bit widths, so negative numbers can't be masked directly the way they are in a native 64-bit integer type) — e.g. `x + (1n << 63n)` before packing, and the inverse after unpacking. That logic is easy to get subtly wrong (sign handling, mask width, a stray off-by-one in the shift) and harder to verify by reading than a string split.
- **Where it would matter:** workloads with millions of live cells per generation, where the constant-factor difference in key construction/hashing compounds across a hot loop.

For a fixed 10-generation run, the live-cell counts involved are small enough that this constant factor is very unlikely to be the bottleneck, and a stray sign/mask bug in the packing logic is a worse risk than the performance it would buy back. This is the kind of optimization to reach for after profiling shows key construction/hashing as a hot spot, not before.

Verdict: *Not adopted; noted as the natural next optimization if profiling on a much larger workload showed key encode/decode or hashing as a bottleneck.*

4. Arbitrary precision

Coordinates are `bigint` throughout. JavaScript's `number` type is an IEEE-754 double, which represents integers exactly only up to `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$, about $9 * 10^{15}$) — roughly three orders

of magnitude short of the signed 64-bit range this problem allows (about $9.2 \cdot 10^{18}$). Past that point, `number` arithmetic silently loses precision: two distinct large coordinates can round to the same `number`, and `x + 1` can equal `x`. There is no exception or visible signal when this happens — it would corrupt coordinates and neighbor calculations invisibly.

`bigint` is native to JavaScript (no third-party big-integer library needed), has arbitrary width, and every neighbor-offset computation (`x + dx`, `y + dy`) is done in `bigint` arithmetic, so precision is never at risk regardless of how close a coordinate is to the edge of the declared range.

The cost is constant-factor: `bigint` arithmetic is slower than native `number` arithmetic (arbitrary-precision values aren't representable in a single machine word/register), and every coordinate consumes more memory than a fixed-width integer would. For this problem — correctness over the full stated range, with a bounded 10-generation run — that cost is clearly worth paying; it would be worth reconsidering only if profiling showed `bigint` arithmetic itself as the bottleneck on a much larger workload, at which point a typed-array-based fixed-width 64-bit representation (e.g. two `BigInt64Array` lanes, or manual high/low 32-bit word arithmetic) would be the next step, at a real cost in implementation complexity.

Verdict: *`bigint` end-to-end — correctness at the stated coordinate range is non-negotiable for this problem, and the performance cost is not the bottleneck at this problem's scale.*

5. Asymptotic complexity

Let `L` be the number of live cells in a generation and `G` be the number of generations (10, fixed by the problem).

- **Time:** each generation is $O(L)$ — every live cell contributes exactly 8 increments to a neighbor-count tally, and the tally (at most $8L$ entries) is walked once to decide the next generation. Total: $O(G \cdot L_{\max})$, where $L_{\max}$ is the largest live-cell count reached across the run.
- **Space:** $O(L)$ for the live set plus $O(L)$ for the neighbor tally per generation. Nothing is ever sized by the coordinate range itself — the same asymptotics hold whether cells are clustered at the origin or scattered across the full int64 range.

This does not bound `L` itself: a pattern can grow between generations (e.g. an exploding pattern unbounded from any direction), and in the worst case `L` could grow geometrically with `G`. For the fixed 10-generation run this problem specifies, that growth is bounded by construction (10 doublings of even a large starting population is still tractable), so it is not a practical concern here — it would become one if the generation count were a large, user-supplied parameter instead of a fixed constant.

This is not claimed to be an asymptotic lower bound for the problem in general (a lower bound would require an argument that no algorithm can do better, which isn't attempted here) — it is simply what this implementation achieves: work proportional to the live population being simulated, not to the size of the coordinate space it's embedded in.

Verdict: *$O(L)$ time and space per generation; not a claimed lower bound, but sufficient that cost tracks the problem's live population rather than its (enormous, mostly irrelevant) coordinate domain.*

6. Deterministic serialization

`serializeLife106` sorts live cells by `(x, y)` before writing them out (`src/serializer.ts`). Set iteration order in JavaScript follows insertion order, which is well-defined but is an accident of how cells happened to be inserted (parse order, then whatever order `step()`'s internal `Map` happened to iterate) — not a property of the board itself.

- **Without sorting:** two runs over the same logical board (e.g. the same cells parsed from files with lines in a different order) could produce byte-different but semantically identical output. That's harmless for a human reading the file, but it breaks anything that diffs output, golden-file tests, or reproducible-build expectations.
- **With sorting:** output is a pure function of the live-cell set, independent of how that set was built. The cost is an $O(L \log L)$ sort per output, which is negligible next to the $O(G \cdot L)$ simulation cost it follows.

This project's own tests rely on this: golden-output comparisons (e.g. the CLI test against the spec's sample input) assert an exact string, which is only stable because output order is deterministic.

Verdict: *Sort before serializing — the $O(L \log L)$ cost is negligible, and it turns output into a pure function of board state, which both tooling and tests depend on.*

7. Streaming

`main()` (`src/cli.ts`) buffers all of stdin into a string before parsing, and `run()` (`src/simulation.ts`) holds the full live-cell set for the current generation in memory — it does not stream cells through `step()` incrementally, and it does not stream output as cells are decided.

This is a direct consequence of the problem's shape: it asks for a fixed number of generations (10) and a single final board printed at the end, not a running commentary on intermediate generations or unbounded simulation. Given that framing, there's no natural place to start streaming output before the final generation is known, and every generation's `step()` call needs the complete previous generation's live set to compute neighbor counts correctly — a cell's fate can depend on a live neighbor added anywhere in the input, so partial/incremental processing of the input can't safely produce partial output.

Streaming would become relevant under different problem constraints:

- An input too large to hold in memory at once — would need a streaming parser that processes the input in a bounded-memory pass (e.g. spatial partitioning of live cells into chunks with overlap at chunk boundaries), independent of the generation-count question.
- A very large or unbounded generation count, or a request for intermediate-generation output — would need output streamed generation-by-generation rather than buffered until the end, and possibly a persistence layer for the live set rather than an in-memory `Set`.
- An explosively growing pattern run for many generations — would need the live-set growth itself bounded or externally stored, independent of the streaming-vs-buffering question for I/O.

Verdict: *Buffered I/O and an in-memory live set, appropriate for a fixed 10-generation run with a single final-board output; would need to be revisited under a much larger generation count or input size.*

8. Boundary semantics

The problem states that *input* coordinates are within the signed 64-bit range, and the parser enforces exactly that: any coordinate outside `[-2^63, 2^63 - 1]` is rejected with an error (`src/parser.ts`).

The problem does not say what should happen when a live cell sits at the boundary and a neighbor computation would step outside it — e.g. a live cell at `x = 9223372036854775807` (int64 max) has a right-hand neighbor at `x = 9223372036854775808`, one past the range. Three behaviors are possible:

- **Wrap (torus topology):** the neighbor coordinate wraps to `x = -9223372036854775808` (int64 min), as if the board were a torus. This invents topology the problem never describes, and would silently connect two coordinates that have no reason to interact from the problem's framing.
- **Clamp/saturate:** the neighbor coordinate is capped at the boundary. This invents an edge that behaves like a wall, again not described by the problem, and creates a discontinuity (cells near the boundary behave differently from cells away from it) that a signed-64-bit-as-arbitrary-space framing doesn't suggest.
- **Let it extend past int64 (this implementation's choice):** since simulation state is `bigint`, not a fixed-width integer, there's no representational reason to stop at `2^63` internally — the range only matters for what counts as valid *input*. A cell born just past the boundary during simulation is simulated normally on subsequent generations.

The chosen behavior only has any effect for inputs deliberately placed at the extreme edge of the range — for the overwhelming majority of inputs (including the problem's own sample, which uses coordinates around $2 * 10^{12}$, nowhere near $9.2 * 10^{18}$) it is unobservable. It's flagged here because it's a real semantic choice, not because it's likely to matter in practice.

Verdict: *Internal (post-parse) arithmetic is unbounded `bigint` math with no wrap/clamp; only parsed input is constrained to signed 64-bit. Wrapping was rejected as inventing unstated torus topology; clamping was rejected as inventing an unstated wall.*

9. Parser leniency

The Life 1.06 parser (`src/parser.ts`) treats the `#Life 1.06` header as optional: it is checked only when line 1 begins with `#`, and a file with no header line at all still parses successfully (every line is simply read as a coordinate pair). A mismatched header (e.g. `#Life 1.05`) is rejected; a missing header is not.

	Strict (header required)	Lenient (header optional, this implementation)
Malformed/misspelled header	Rejected	Rejected (unchanged)
No header line at all	Rejected	Accepted — parsed as coordinate pairs
Failure mode for a truncated/edited file that lost its header	Hard failure, no output	Still simulates correctly

The parser's actual job is reading coordinate pairs; the header is a label on the format, not information the simulation needs. Rejecting a well-formed coordinate list purely because a header line is missing would be enforcing a formatting nicety at the cost of otherwise-valid input — the kind of validation that protects against a scenario (a hand-edited or header-stripped file) more than it protects against a real error. A misspelled or wrong-version header (`#Life 1.05`) is different: that's a positive signal something is actually wrong (wrong format version, or accidental concatenation of two files), so it is still rejected.

Verdict: *Reject a wrong header, accept a missing one — validate the signal that indicates an actual problem, not the presence of an optional label.*